

Scratch Finder 공정 혐의점수 로직 보고서

wafer 단위 실제 Good/Bad 상태와 별도 정황 변수 y 를 결합한 process_id별 의심도 산정 방법

초록

본 보고서는 Scratch Finder의 wafer-level 공정 혐의점수 산정 로직을 논문형 구조로 정리한다. 각 wafer의 실제 상태 z in $\{G, B\}$ 와 외부 로직에서 계산된 정황 y in $\{0, 1\}$ 를 입력으로 받아, Bad wafer가 $y=1$ 정황에 우연 이상으로 집중되는지를 hypergeometric tail probability로 검정하고, Bad miss와 Good false context에 대한 비대칭 품질 페널티 및 표본 수 보정을 곱해 최종 score를 계산한다.

핵심 설계 원칙은 세 가지이다. 첫째, 실제 Bad가 $y=0$ 에 놓이는 경우를 강하게 벌점화한다. 둘째, $y=1$ 에 Bad가 몰리는 통계적 집중도만 evidence로 인정하여 all- $y1$ /all- $y0$ /no-contrast 케이스를 0점 처리한다. 셋째, 작은 N 에서 우연히 완벽히 맞은 경우가 과대평가되지 않도록 $r_N = N / (N + k_N)$ 을 적용한다.

1. 문제 설정

입력 데이터는 process_id, wafer_id, state, y 네 개의 필수 컬럼을 갖는다. state는 실제 wafer 상태로 G 또는 B이며, y 는 별도 로직이 산출한 정황 변수로 1은 bad context, 0은 good context를 의미한다. 공정별 관측치는 다음 2x2 count table로 압축된다.

	$y=1$	$y=0$	합계
state=B	n_{B1}	n_{B0}	n_B
state=G	n_{G1}	n_{G0}	n_G
합계	m	$N-m$	N

정상적인 혐의 패턴은 Bad wafer가 $y=1$ 에, Good wafer가 $y=0$ 에 놓이는 것이다. 반대로 Bad가 $y=0$ 이면 강한 miss penalty가 발생하고, Good이 $y=1$ 이면 상대적으로 약한 false context penalty가 발생한다.

2. 비대칭 likelihood와 품질 항

```
p_B = P(y=1 | state=B), default 0.95
p_G = P(y=1 | state=G), default 0.20
```

```
log_likelihood =
  n_B1 * log(p_B)
+ n_B0 * log(1 - p_B)
+ n_G1 * log(p_G)
+ n_G0 * log(1 - p_G)
```

```
mean_likelihood = exp(log_likelihood / N)
```

mean_likelihood는 모델 적합도를 직관적으로 보는 진단값이다. 최종 ranking에는 단독 사용하지 않는다. ranking의 품질 항은 miss/false-context rate를 상태별로 정규화한 balanced penalty로 정의한다.

```
w_B0 = log(p_B / (1 - p_B))
w_G1 = log((1 - p_G) / p_G)

bad_miss_rate = n_B0 / n_B, if n_B > 0 else 0
good_false_context_rate = n_G1 / n_G, if n_G > 0 else 0

J_bal =
  alpha_B * w_B0 * bad_miss_rate
+ alpha_G * w_G1 * good_false_context_rate

quality = exp(-J_bal)
```

기본값 $\alpha_B=0.7$, $\alpha_G=0.3$ 은 Bad miss를 Good false context보다 더 중요하게 본다는 도메인 가정을 반영한다. $p_B=0.95$, $p_G=0.20$ 이면 w_{B0} 는 w_{G1} 보다 크며, Bad가 $y=0$ 에 놓이는 패턴이 강하게 감점된다.

3. 통계적 evidence

evidence는 $y=1$ 정황 m 개가 전체 N 개 wafer 중 무작위로 선택되었다는 귀무가설 아래, Bad wafer가 관측치 n_{B1} 개 이상 포함될 확률의 음의 로그값이다.

```
X ~ Hypergeometric(N, n_B, m)

p_tail = P(X >= n_B1)
        = sum_{x=n_B1}^{min(n_B, m)}
          C(n_B, x) * C(n_G, m - x) / C(N, m)

evidence = -log(max(p_tail, eps))
```

이 항은 $y=1$ 집합 내부에 Bad가 얼마나 우연 이상으로 몰렸는지를 측정한다. count-level probability에는 조합항을 포함하며, SciPy가 있으면 `scipy.stats.hypergeom.sf`를 사용하고 없으면 lgamma 기반 `log_comb` fallback으로 계산하도록 구현되어 있다.

4. Degenerate 및 no-contrast 처리

다음 조건은 통계적 contrast가 없으므로 $p_{tail}=1$, $evidence=0$, $score=0$ 으로 처리한다.

- $N == 0$: 관측 wafer가 없음
- $n_B == 0$ 또는 $n_G == 0$: 한쪽 실제 상태 class가 없음
- $m == 0$: $y=1$ 정황이 없음
- $m == N$: 모든 wafer가 $y=1$ 이라 정황이 구분력을 갖지 않음

이 처리는 `all_y1_N25`, `all_y0_N25`, `no_bad_N25`, `no_good_N5` 같은 케이스가 잘못 높은 점수를 받는 것을 막는다.

5. 최종 score와 참고 exact-alt score

```
r_N = N / (N + k_N), default k_N = 8

score = r_N * quality * evidence

log_count_prob_alt =
    log C(n_B, n_B1)
  + n_B1 * log(p_B)
  + n_B0 * log(1 - p_B)
  + log C(n_G, n_G1)
  + n_G1 * log(p_G)
  + n_G0 * log(1 - p_G)

score_exact_alt = r_N * quality * (-log(max(count_prob_alt, eps)))
```

기본 ranking은 score를 사용한다. `score_exact_alt`는 동일 입자 문제가 아니라 count-level event probability를 보고 싶을 때의 참고값이다. 따라서 최종 혐의점수는 적합도 자체가 아니라, contrast가 있는 통계적 집중도와 도메인 품질, 표본 수 안정성을 곱한 값이다.

6. 시뮬레이션 결과

아래 표는 기본 파라미터 $p_B=0.95$, $p_G=0.20$, $\alpha_B=0.7$, $\alpha_G=0.3$, $k_N=8$, $\epsilon=1e-12$ 로 11개 시뮬레이션 케이스를 score 내림차순 정렬한 결과이다.

rank	process_id	N	n_B	n_G	m	quality	evidence	r_N	score	exact_alt
1	lot_perfect_N25	25	5	20	5	1	10.88	0.7576	8.243	3.575
2	noisy_good_in_y1_N25	25	5	20	13	0.8467	3.72	0.7576	2.387	2.608
3	medium_perfect_N10	10	2	8	2	1	3.807	0.5556	2.115	1.049
4	bad_missed_N25	25	5	20	1	0.1923	1.609	0.7576	0.2344	2.168
5	random_mix_N25	25	5	20	6	0.2672	1.069	0.7576	0.2163	1.682
6	tiny_perfect_N2	2	1	1	1	1	0.6931	0.2	0.1386	0.05489
7	tiny_single_bad_N1	1	1	0	1	1	0	0.1111	0	0.005699
8	all_y1_N25	25	5	20	25	0.6598	0	0.7576	0	13.81
9	all_y0_N25	25	5	20	0	0.1273	0	0.7576	0	1.875
10	no_bad_N25	25	0	25	0	1	0	0.7576	0	4.226
11	no_good_N5	5	5	0	5	1	0	0.3846	0	0.09864

lot_perfect_N25는 $N=25$ 에서 Bad 5개가 모두 $y=1$ 이고 Good 20개가 $y=0$ 이므로 evidence와 quality가 모두 높다. medium_perfect_N10과 tiny_perfect_N2도 완벽하지만 r_N 과 hypergeometric evidence가 작아 순위가 낮아진다. noisy_good_in_y1_N25는 Bad를 놓치지 않는 않지만 Good 8개가 $y=1$ 에 들어와 quality가 감소한다. bad_missed_N25는 Bad 4개가 $y=0$ 에 놓여 강한 miss penalty를 받는다.

추가 진단 컬럼

process_id	n_B1	n_B0	n_G1	n_G0	mean_like	p_tail
lot_perfect_N25	5	0	0	20	0.828	1.88e-05
noisy_good_in_y1_N25	5	0	8	12	0.5313	0.02422
medium_perfect_N10	2	0	0	8	0.828	0.02222
bad_missed_N25	1	4	0	20	0.5169	0.2
random_mix_N25	2	3	4	16	0.4658	0.3434
tiny_perfect_N2	1	0	0	1	0.8718	0.5
tiny_single_bad_N1	1	0	0	0	0.95	1
all_y1_N25	5	0	20	0	0.2731	1
all_y0_N25	0	5	0	20	0.4595	1
no_bad_N25	0	0	0	25	0.8	1
no_good_N5	5	0	0	0	0.95	1

7. 시각적 검증

각 subplot은 score 기준 내림차순으로 정렬된 process_id별 wafer 배치를 나타낸다. x축은 wafer_id 순서, y축은 정황 y이며, 파란 원은 Good wafer, 빨간 X는 Bad wafer를 의미한다.

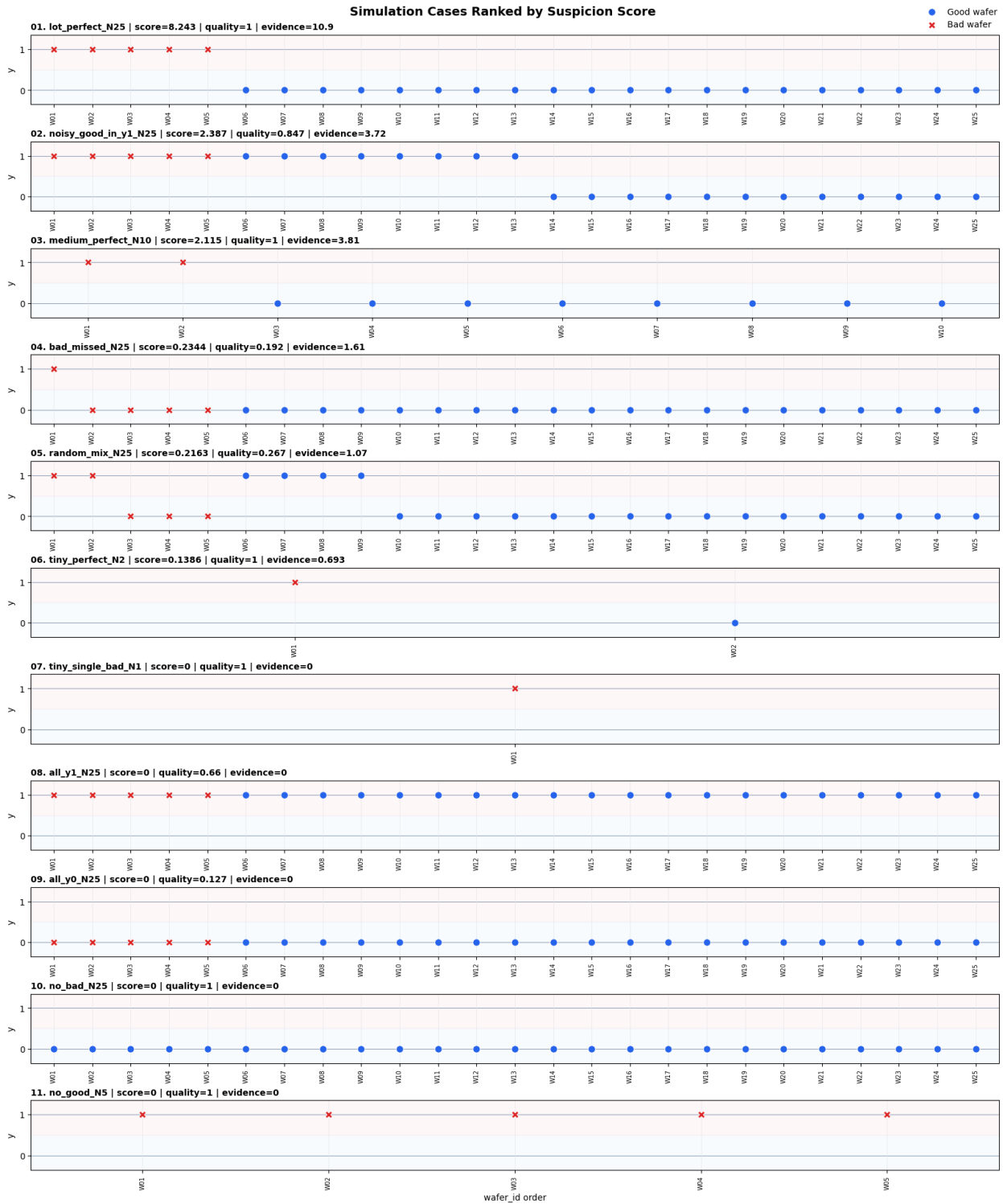


Figure 1. Simulation cases ranked by suspicion score. 제목에는 rank, process_id, score, quality, evidence가 표시된다.

8. 구현 구조

함수	역할
log_comb(n, k)	lgamma 기반 $\log C(n, k)$ 계산으로 overflow를 회피한다.
hypergeom_tail_pvalue	Bad 집중도에 대한 hypergeometric upper-tail p-value를 계산한다.
score_process_counts	2x2 count를 받아 likelihood, quality, evidence, score 전체 dict를 반환한다.
score_process_dataframe	wafer-level DataFrame을 process_id별로 groupby하여 score table을 만든다.
make_case / make_sim_df	지정된 count 조합에서 wafer-level simulation DataFrame을 생성한다.
plot_ranked_cases	score 순위대로 y-context scatter plot을 생성한다. output_path가 없으면 저장하지 않는다.
run_simulation_cases	시뮬레이션 표 출력, sanity check, 선택적 plot 저장을 수행한다.

Sanity check

- `score(lot_perfect_N25) > score(medium_perfect_N10) > score(tiny_perfect_N2)`
- `score(lot_perfect_N25) > score(noisy_good_in_y1_N25) > score(bad_missed_N25)`
- `score(all_y1_N25), score(all_y0_N25), score(no_bad_N25), score(no_good_N5)`는 0에 가깝다.

9. 해석 및 한계

본 score는 원인 확률 그 자체가 아니라, 관측된 y-context가 Bad wafer를 얼마나 통계적으로 분리하고 있는지에 대한 우선순위 지표이다. 따라서 score가 높다는 것은 해당 process_id가 후속 공정/설비/시간대 분석에서 먼저 조사될 후보라는 뜻이지, 단독으로 인과를 확정한다는 의미는 아니다.

파라미터 p_B , p_G , α_B , α_G , k_N 는 도메인 비용 구조와 데이터 규모에 따라 보정될 수 있다. 예를 들어 false alarm 비용이 큰 분석 환경에서는 α_G 를 높이고, 작은 lot에서 더 보수적으로 보고 싶으면 k_N 을 키울 수 있다.

10. 결론

Scratch Finder 로직은 비대칭 likelihood 모델, balanced quality penalty, hypergeometric evidence, sample-size shrinkage를 결합하여 wafer-level 정황 y와 실제 Good/Bad 상태 사이의 공정별 이상 집중도를 산정한다. degenerate/no-contrast 케이스를 명시적으로 0점 처리하고 작은 N의 과대평가를 완화하므로, lot 단위 또는 process_id 단위 후보 랭킹에 실용적으로 사용할 수 있다.

부록: 사용 파일

- `wafer_process_suspicion_sim.py`: scoring 함수, simulation case, plotting 함수 구현
- `wafer_process_suspicion_sim.ipynb`: 표 display와 inline plot 실행 예시
- `output/pdf/scratch_finder_logic_report.pdf`: 본 보고서